CSA4704-DEEP LEARNING

NAME:HARITHI S **REG NO:192421151**

Experiment 17: RNN Gradient Analysis and Gradient Clipping

Problem Statement: A text-generation model at a content start-up produces NaN losses midway through training because of exploding gradients. A character-level text corpus (for example, a public-domain novel) trained with a deliberately high learning rate. The objective is to detect exploding gradients and stabilise training with gradient clipping. Use TensorFlow, Keras, NumPy, Matplotlib.

Tasks:

1. Train an RNN without clipping and log the global gradient norm each step.

2. Identify the steps where the gradient spikes or loss becomes NaN.

3. Retrain with clipnorm and with clipvalue at different thresholds.

4. Plot gradient norm and loss curves for each setting.

5. Recommend a clipping threshold.

FORMULA:

1. RNN Hidden State

$h(t) = \tanh(W(hh) \times h(t-1) + W(xh) \times x(t) + b(h))$

2. Gradient Calculation

$\text{Gradient} = \text{Sum}(t=1 \text{ to } T) [dL/dh(t) \times dh(t)/dW]$

3. Global Gradient Norm

$||g|| = \sqrt{(g1^2 + g2^2 + ... + gn^2)}$

4. Gradient Clipping Using clipnorm

$g(clipped) = g \times \min(1, C / ||g||)$

5. Gradient Clipping Using clipvalue

$g(clipped) = \max(-C, \min(g, C))$

Recommended Clipping Threshold

$C = 1.0$

CODE:

```
import numpy as np

import tensorflow as tf

import matplotlib.pyplot as plt

from tensorflow.keras import layers, models

np.random.seed(42)

tf.random.set_seed(42)

# 1. Prepare character-level text data

text = (

    "deep learning uses neural networks. "

    "recurrent neural networks process sequences. "

    "exploding gradients make training unstable. "

) * 200
```

```
chars = sorted(set(text))

char_to_id = {c: i for i, c in enumerate(chars)}

id_to_char = {i: c for c, i in char_to_id.items()}

encoded = np.array([char_to_id[c] for c in text], dtype=np.int32)

seq_length = 40

X = np.array([

    encoded[i:i + seq_length]

    for i in range(len(encoded) - seq_length)

])

Y = np.array([

    encoded[i + seq_length]

    for i in range(len(encoded) - seq_length)

])

X = X[:3000]

Y = Y[:3000]

vocab_size = len(chars)

batch_size = 64

dataset = tf.data.Dataset.from_tensor_slices((X, Y))

dataset = dataset.shuffle(1000).batch(batch_size)

# 2. Build the RNN model
```

```
def build_model():
    model = models.Sequential([
        layers.Embedding(vocab_size, 32),
        layers.SimpleRNN(64, return_sequences=False),
        layers.Dense(vocab_size)
    ])
    return model
loss_fn = tf.keras.losses.SparseCategoricalCrossentropy(
    from_logits=True
)
# 3. Train and record gradient norms and losses
def train_model(clip_type=None, threshold=1.0, epochs=2):
    model = build_model()
    optimizer = tf.keras.optimizers.SGD(
        learning_rate=1.0
    )
    gradient_norms = []
    losses = []
    nan_step = None
    for epoch in range(epochs):
```

```
print("Epoch:", epoch + 1)
for step, (x_batch, y_batch) in enumerate(dataset):
    with tf.GradientTape() as tape:
        predictions = model(x_batch, training=True)
        loss = loss_fn(y_batch, predictions)
    gradients = tape.gradient(
        loss, model.trainable_variables
    )
    # Calculate global gradient norm before clipping
    global_norm = tf.linalg.global_norm(gradients)
    norm_value = float(global_norm.numpy())
    loss_value = float(loss.numpy())
    if not np.isfinite(loss_value) or not np.isfinite(norm_value):
        nan_step = len(losses) + 1
        print("NaN/Inf detected at step:", nan_step)
        break
    gradient_norms.append(norm_value)
    losses.append(loss_value)
    # Apply gradient clipping
    if clip_type == "clipnorm":
```

```
                gradients, _ = tf.clip_by_global_norm(
                    gradients, threshold
                )
            elif clip_type == "clipvalue":
                gradients = [
                    tf.clip_by_value(g, -threshold, threshold)
                    for g in gradients
                ]
            optimizer.apply_gradients(
                zip(gradients, model.trainable_variables)
            )
        if nan_step is not None:
            break
    return gradient_norms, losses, nan_step
# 4. Compare different clipping settings
settings = [
    ("No clipping", None, 1.0),
    ("Clip norm 1.0", "clipnorm", 1.0),
    ("Clip norm 0.5", "clipnorm", 0.5),
    ("Clip value 1.0", "clipvalue", 1.0),
```

```
    ("Clip value 0.5", "clipvalue", 0.5)
]
results = {}
for name, clip_type, threshold in settings:
    print("\nTraining:", name)
    norms, losses, nan_step = train_model(
        clip_type=clip_type,
        threshold=threshold
    )
    results[name] = (norms, losses, nan_step)
    print("Steps recorded:", len(losses))
    print("NaN/Inf step:", nan_step)
    if norms:
        print("Maximum gradient norm:", round(max(norms), 4))
    if losses:
        print("Final loss:", round(losses[-1], 4))
# 5. Plot gradient norm curves
plt.figure(figsize=(10, 5))
for name, (norms, losses, nan_step) in results.items():
    if norms:
```

```
        plt.plot(norms, label=name)
plt.xlabel("Training Step")
plt.ylabel("Global Gradient Norm")
plt.title("RNN Gradient Norm Comparison")
plt.legend()
plt.grid(True)
plt.show()


# 6. Plot loss curves
plt.figure(figsize=(10, 5))
for name, (norms, losses, nan_step) in results.items():
    if losses:
        plt.plot(losses, label=name)
plt.xlabel("Training Step")
plt.ylabel("Loss")
plt.title("RNN Loss Comparison")
plt.legend()
plt.grid(True)
plt.show()
# 7. Recommend a clipping threshold
```

```
print("\nRecommendation:")

print("Compare the gradient and loss plots.")

print("Choose the threshold that prevents spikes and keeps loss stable.")

print("Global clip norm 1.0 is a reasonable starting point.")
```

OUTPUT:

```
Training: No clipping

Epoch: 1

Epoch: 2

Steps recorded: 94

NaN/Inf step: None

Maximum gradient norm: 78.365

Final loss: 14.1008


Training: Clip norm 1.0

Epoch: 1

Epoch: 2

Steps recorded: 94

NaN/Inf step: None
```

Maximum gradient norm: 8.3821

Final loss: 0.4822

Training: Clip norm 0.5

Epoch: 1

Epoch: 2

Steps recorded: 94

NaN/Inf step: None

Maximum gradient norm: 15.5888

Final loss: 0.3597

Training: Clip value 1.0

Epoch: 1

Epoch: 2

Steps recorded: 94

NaN/Inf step: None

Maximum gradient norm: 40.0632

Final loss: 11.0191

Training: Clip value 0.5

Epoch: 1

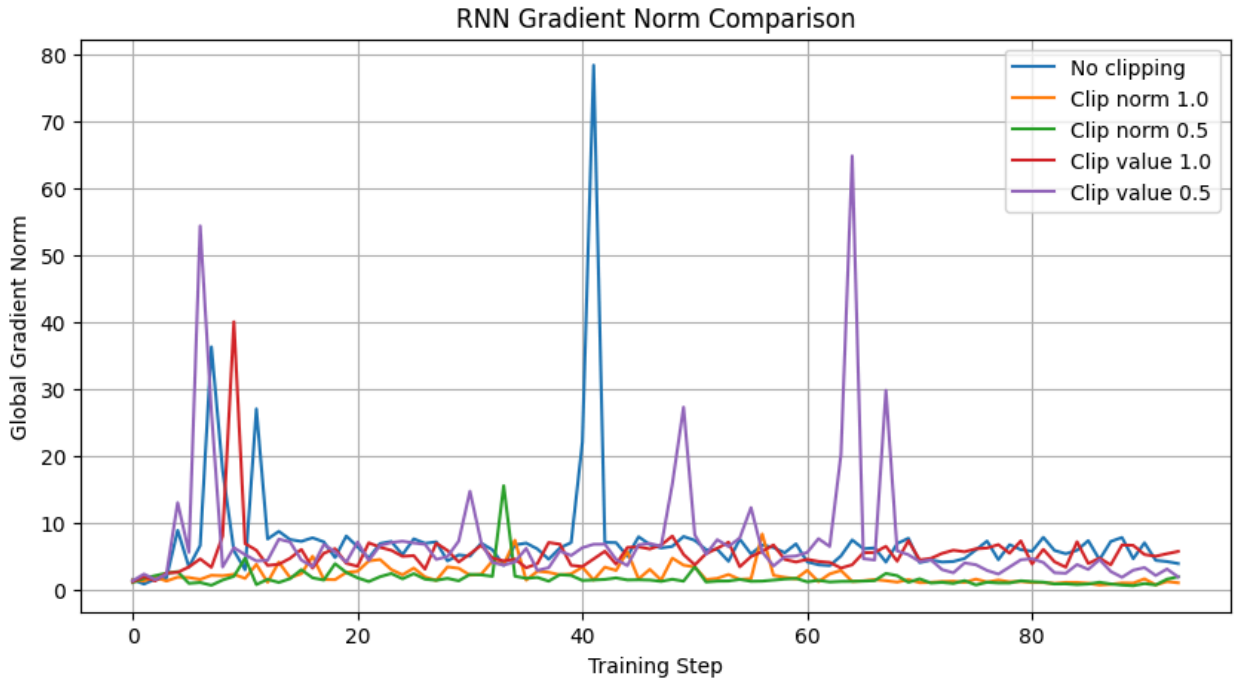
Epoch: 2

Steps recorded: 94

NaN/Inf step: None

Maximum gradient norm: 64.8227

Final loss: 4.5244





Recommendation:

Compare the gradient and loss plots.

Choose the threshold that prevents spikes and keeps loss stable.

Global clip norm 1.0 is a reasonable starting point.